

ML Machine Coding Assignment

Personalized Astrology Chat — Shared Brain & Memory

Time Limit: 3 Hours

Experience: SDE2 / 3–5 Years

Language: Python

Framework: FastAPI preferred

LLM: Any provider

Database: Neo4j preferred; other graph databases are allowed



Problem Statement

MyNaksh is building a personalized astrology platform where the AI should understand not only the current question, but also relevant information about the user.

Build a conversational AI service that:

- Accepts a user's profile and chat messages.
- Maintains short-term conversation context.
- Builds a persistent **Shared Brain** containing useful user information.
- Retrieves relevant information from the Shared Brain when answering questions.
- Uses an LLM to generate personalized responses.
- Persists useful information that can improve future conversations.

The goal is to demonstrate how you would build a **production-oriented conversational ML system**.

You do **not** need to build a complete astrology engine.



Example Conversation

First conversation

User:

My name is Rahul. I was born on 15 August 1995 in Delhi. I'm planning to switch jobs next year.

The system should identify information that may be useful in future conversations.

For example:

User

└─ HAS_GOAL → Career Change

└─ target_year → 2027

Later in the same conversation

User:

What should I focus on for my career?

The system should use the relevant user context to personalize the response.

Follow-up

User:

Why do you say that?

The system should understand this using the current conversation context.

New conversation

User:

What do you remember about my career goals?

The system should retrieve the relevant long-term information from the Shared Brain.

1. Chat API

Implement a simple API such as:

POST /chat

Request

```
{  
  "user_id": "user-123",  
  "session_id": "session-456",  
  "message": "What should I focus on in my career?"
```

```
}
```

You may optionally accept profile information during user creation or through a separate endpoint.

Response

```
{  
  "response": "Based on your career goals...",  
  "user_id": "user-123",  
  "session_id": "session-456",  
  "context_used": [  
    "career_goal",  
    "user_profile"  
  ]  
}
```

The response format may be extended where useful.

2. User Profile

Support basic user information such as:

- Name
- Date of birth
- Time of birth
- Birth place
- Preferred language
- Zodiac / Sun Sign

You may use simplified or stubbed astrology calculations.

The focus is on **using user information in the conversational system**, not implementing a full astrology computation engine.

3. Shared Brain

Build a persistent **Shared Brain / User Intelligence layer**.

It should be capable of storing useful long-term information such as:

- User profile
- Goals
- Preferences
- Interests
- Important memories
- Important life areas
- Astrology attributes

Example:

User

|— HAS_GOAL → Career Growth
|— INTERESTED_IN → Entrepreneurship
|— PREFERS → Hindi
└— HAS_ZODIAC → Leo

Storage

Use **Neo4j or another graph-based storage solution**.

Neo4j is preferred, but you may use another graph database or lightweight graph implementation if you can justify the choice.

You should design the graph/schema yourself.

Briefly explain:

- What entities you store.
- What relationships you create.
- How information is retrieved.
- Why you chose your approach.

4. Memory

The system should distinguish between **short-term conversation context** and **long-term memory**.

Short-Term Context

Used to understand the current conversation.

Example:

"Why do you say that?"

This should primarily use recent conversation context.

Long-Term Memory

Information that may remain useful across future conversations.

Example:

"I'm planning to switch jobs next year."

This could become a persistent career goal.

The system should **not store every message as long-term memory**.

Implement a simple mechanism to identify useful information worth storing.

5. Context Selection

When a user sends a message, the system should select relevant context before calling the LLM.

For example:

User Message



Understand Query



Select Relevant User Context



Build LLM Context



Generate Response



Update Memory

The system should avoid sending the entire user graph and entire conversation history to the LLM.

Only relevant information should be included.

The implementation can use:

- Rules
- LLM-based extraction/selection
- A simple classifier
- Hybrid approach

The choice is up to you.

6. LLM Layer

Use an LLM through a clean abstraction.

You may use any suitable provider, such as:

- OpenAI
- Gemini
- Anthropic
- Mistral
- Open-source/local model

The LLM should receive an appropriate combination of:

- Current user message
- Recent conversation context
- Relevant Shared Brain information
- User profile / astrology information

Keep the LLM integration modular so that the provider can be changed without rewriting the application.

7. Memory Update

After responding to the user, the system should be capable of identifying information that may be useful for future conversations.

For example:

User:

I'm preparing for a product management interview next month.

Potential memory:

User

└─ HAS_GOAL

└─ Product Management Interview

└─ timeframe → Next Month

The implementation does not need to be complex.

We want to see how you think about:

- What should be remembered?
- What should not be remembered?
- How should memories be represented?
- How should existing memories be updated?

8. Testing & Evaluation

Create **5–10 test scenarios** covering cases such as:

1. New user
2. Creating a long-term memory
3. Retrieving a memory
4. Follow-up question
5. New session using previous information
6. Irrelevant memory
7. User correcting existing information
8. Missing user information

Include basic automated tests.

Briefly explain how you would evaluate whether the Shared Brain is improving the quality of responses.

Consider areas such as:

- Memory accuracy
- Context relevance
- Personalization
- Conversation consistency
- Irrelevant context

- Memory persistence

A sophisticated evaluation framework is **not required**.

9. Error Handling

Handle reasonable failure scenarios such as:

- Invalid input
- Missing profile information
- LLM failure
- Database/graph failure
- Empty memory
- No relevant context

Provide sensible fallbacks.

Suggested Project Structure

You may structure the project however you prefer.

A possible structure:

app/

├── api/

├── chat/

├── llm/

├── memory/

├── brain/

├── profile/

└── tests/

README.md

requirements.txt

This is only a suggestion. You are free to choose your own architecture.

Optional / Bonus

Only attempt these after the core flow is working:

- Memory importance/confidence score
- Memory conflict resolution
- Memory expiration/decay
- Conversation summarization
- Model fallback/routing
- Context/token optimization
- Hindi/multilingual responses
- Advanced graph traversal

Submission

Please submit:

1. Zip or GitHub repository
2. Working application
3. README containing:
 - Architecture
 - Shared Brain schema
 - Memory strategy
 - Context-selection approach
 - Key design decisions/trade-offs
 - Production considerations
4. Basic automated tests
5. Sample API requests/responses

The assignment is designed to be completed in **3 hours**.

Priority

If time is limited, prioritize this end-to-end flow:

Chat → Context Selection → Shared Brain → LLM → Response → Memory Update

A complete, well-designed core implementation is preferred over many unfinished bonus features.